

FinGuard External — Desafio de ML com Dados Sintéticos

O Problema

Instituições financeiras recebem milhares de reclamações de clientes por dia — via SAC, ouvidoria, redes sociais e Banco Central. Esse volume é impossível de processar manualmente com qualidade e velocidade.

O desafio é construir um pipeline de Machine Learning que vá além da classificação simples: **descobrir padrões ocultos nas reclamações que analistas humanos não conseguiriam identificar**, usando técnicas não-supervisionadas.

Parte 1 — Geração de Dados Sintéticos

Antes de qualquer análise, você precisa criar o dataset. Não usaremos dados reais de clientes — você vai **gerar um dataset sintético realista** de reclamações financeiras.

O que gerar

Um CSV com **500 reclamações fictícias** contendo os seguintes campos:

Campo	Descrição	Exemplo
id	Identificador único	REC-2026-00142
data_reclamacao	Data do registro	2026-01-15
canal	Canal de origem	SAC, Ouvidoria, Banco Central, Redes Sociais
texto_reclamacao	Texto livre da reclamação	"Fui cobrado duas vezes na fatura..."
produto	Produto envolvido (pode ser vazio)	Cartão de Crédito, Conta Corrente, Empréstimo
status	Status atual	Aberta, Em análise, Resolvida

Requisitos de qualidade do dataset sintético

O dataset **deve ser realista e desafiador**. Para isso:

- **Variação de tom:** neutro, irritado, formal, informal, crítico
- **Variação de complexidade:** reclamações simples ("cobrança errada") até indícios de fraude
- **Dados imperfeitos:** algumas reclamações sem campo `produto`, erros de digitação, palavrões, gírias
- **Distribuição realista de categorias:**
 - Cobrança Indevida: ~30%
 - Atendimento: ~20%
 - Fraude/Segurança: ~15%
 - Produto/Serviço: ~20%
 - Cancelamento: ~10%
 - Outros: ~5%
- **Pelo menos 50 reclamações com indício de risco alto** (menção a Banco Central, fraude, processo judicial)
- **Pelo menos 20 reclamações com dados sensíveis** embutidos no texto (CPF, número de conta)

Como gerar

Use um LLM (via API da sua preferência — OpenAI, Anthropic, Gemini) para gerar os textos das reclamações de forma programática. O script de geração deve:

1. Receber como input a distribuição desejada de categorias
2. Gerar textos variados em PT-BR com tom e complexidade variáveis
3. Montar o CSV final com todos os campos
4. Documentar o processo de geração no repositório

Parte 2 — Pipeline de ML Não-Supervisionado

Com o dataset gerado, construa um pipeline completo de descoberta de padrões.

Objetivo

Descobrir agrupamentos naturais nas reclamações **sem usar as categorias pré-definidas** — deixar o algoritmo encontrar os padrões por conta própria. Depois, comparar o que ele descobriu com as categorias que você definiu na geração.

O que entregar

Etapa 1 — Vetorização (Embeddings)

- Gerar embeddings semânticos para cada reclamação usando um modelo de embeddings (ex: `sentence-transformers`, `text-embedding-3-small` da OpenAI, ou similar)
- Documentar o modelo escolhido e por quê
- Tempo de processamento deve ser menor que 10 minutos para o dataset completo

Etapa 2 — Clustering

- Aplicar K-means (ou algoritmo equivalente como DBSCAN ou LDA) sobre os embeddings
- **Justificar o número de clusters escolhido** usando pelo menos um dos métodos:
 - Elbow Method (plotar inércia para $k=2..15$)
 - Silhouette Score (validar coesão dos clusters)
- O Silhouette Score do modelo final deve ser documentado

Etapa 3 — Interpretação

- Para cada cluster descoberto, usar um LLM para:
 - Nomear o cluster semanticamente (ex: "Fraudes em cartão por engenharia social")
 - Listar os 3 temas principais do cluster
 - Identificar o nível de risco predominante

Etapa 4 — Análise Crítica

Comparar os clusters descobertos com as categorias pré-definidas que você usou na geração:

- Os clusters coincidem com as categorias? Onde divergem?
 - O algoritmo descobriu subcategorias que você não havia previsto?
 - Quais reclamações “escaparam” da categorização esperada e por quê?
-

Parte 3 — Integração com Pipeline de Análise

O pipeline de ML não deve existir isolado. Integre os resultados com um sistema básico de análise de reclamações:

- Um agente LLM (via API de sua escolha) que recebe uma reclamação e retorna JSON com: categoria, produto, sentimento, urgência, resumo
 - O cluster descoberto pelo ML é adicionado como campo extra no JSON de saída
 - O relatório final inclui tanto a classificação supervisionada (agente LLM) quanto o cluster não-supervisionado (ML)
-

Entregáveis

1. **Script de geração** do dataset sintético (`generate_dataset.py`)
 2. **Dataset gerado** (`reclamacoes_sinteticas.csv`)
 3. **Pipeline de ML** (`pipeline_ml.py`) com embeddings + clustering + interpretação
 4. **Notebook de análise** com visualizações:
 - Plot do Elbow Method
 - Visualização dos clusters em 2D (UMAP ou t-SNE)
 - Distribuição de categorias por cluster
 5. **Relatório final** em HTML ou Markdown com análise crítica
 6. **README** explicando como rodar tudo do zero
-

Stack sugerida

- Python 3.11+

- **Geração de dados:** LLM via API (OpenAI, Anthropic, Gemini — sua escolha)
 - **Embeddings:** `sentence-transformers` ou API de embeddings
 - **ML:** `scikit-learn` (K-means, métricas), `umap-learn` (visualização)
 - **LLM para interpretação:** mesma API da geração ou diferente
 - **Relatório:** Pandas + Jinja2 para HTML, ou Jupyter Notebook
-

Critérios de Avaliação

Critério	Peso	Descrição
Qualidade dos dados sintéticos	25%	Dataset realista, variado, com dados imperfeitos
Pipeline de ML	30%	Embeddings, clustering, justificativa do K
Análise crítica	25%	Comparação clusters vs categorias, insights gerados
Código e documentação	20%	Reprodutibilidade, clareza, README

Regras

- Proibido usar datasets reais de clientes
- O script de geração deve ser reproduzível (seed fixo para aleatoriedade)
- Todo o pipeline deve rodar localmente do zero com `python pipeline_ml.py`
- Custos de API devem ser estimados e documentados